

JavaScript

Good and Bad Parts

Tvorba internetových aplikací

RNDr. Richard Ostertág, PhD.
KI FMFI UK Bratislava
ostertag@dcs.fmph.uniba.sk

Sources

- Mozilla Developer Network: JavaScript
 - <https://developer.mozilla.org/en-US/docs/JavaScript>
- Douglas Crockford: “JavaScript: The Good Parts”
 - <http://shop.oreilly.com/product/9780596517748.do>
- Annotated ECMAScript 5.1
 - <http://es5.github.com/>
- ECMAScript Language Specification
 - Standard ECMA-262 / 5.1 Edition / June 2011
 - <http://ecma-international.org/ecma-262/5.1/>

Strict mode

- converts mistakes into errors
- simplifying variable uses
 - JavaScript engines can better optimize
- secures JavaScript
- prohibits some syntax likely to be defined in future versions of ECMAScript
- "use strict"; before any statement
 - in script: for entire script
 - in function: only for selected function

Strict mode – syntax example

- ```
function strict() { // function-level strict mode
 "use strict";
 function nested() { return "also strict"; }
 // strict
 return nested();
}
function nonStrict() { return "non strict"; }
```

# Strict mode – differences

- disables with
  - `var x; with (object) { x }`
  - `x` can be bind to global `x` or to `object.x`
- disables creation of global variables
  - `undefinedVar = 0; //` throws a `ReferenceError`
- assignment to a non-writable property throws
  - `var o = {};`  
    `Object.defineProperty(o, "x",`  
        `{ value: 0, writable: false });`  
    `o.x = 1; //` throws a `TypeError`
  - the same for getter-only property
- etc.

# Functions – arguments variable

- the arguments variable
  - a bonus “parameter” available to functions
  - gives the function access to all of the arguments
    - including excess arguments not assigned to parameters
- it’s possible to write functions with variable number of parameters:
- ```
function sum() {  
    var sum = 0;  
    for (var i = 0; i < arguments.length; i += 1)  
        { sum += arguments[i]; }  
    return sum; };
```
- `sum(1, 9, 9, 5, 3) ↦ 27`

Functions – return statement

- can be used to cause the function to return early without executing the remaining statements
- function always returns a value
 - if not specified, then undefined is returned
- invocation with the ***new*** prefix
 - if the return value is not an object
 - then ***this*** is returned instead

Exceptions

- exceptions are unusual but not completely unexpected situations, e.g.:
 - end of file
 - is not nothing unusual
 - should not be an exception
 - disk full
 - is unusual
 - but not completely unexpected
 - this can be exception

Exceptions

- throw statement
 - interrupts execution of the function
 - the statements after throw won't be executed
 - control will be passed to the first catch block in the call stack
 - takes an expression as the only argument
 - ideally an object with a name and message property
 - name: identifies the type of the exception
 - message: describe exception in more details
 - you can also add other properties
- try statement
 - has only single catch block that catches all exceptions
- catch clause
 - the exception object is passed to catch argument
 - exception dependent handling
 - inspect the name property of passed object inside handler

Exceptions

- ```
function add(a, b) {
 if (typeof a !== 'number' || typeof b !== 'number') {
 throw {
 name : 'TypeError',
 message : 'add needs two numbers'
 };
 }
 return a + b;
}
```
- ```
function try_it() {  
  try { add("seven"); }  
  catch (e) {  
    console.log(e.name + ': ' + e.message);  
  }  
}
```
- ```
try_it() ↦ undefined
```

  - `TypeError: add needs two numbers`

# Tail recursion

- tail recursion optimization
  - invocation is replaced with a loop
  - num. of iterations > stack size
  - significantly speed things up
- but JavaScript currently does not provide it:
- ```
function sum(i, a) {  
  a = a || 0;  
  if (i < 1) { return a; }  
  return sum(i - 1, a + i);  
};
```
- `sum(3638) ↦ 6619341`
- `sum(3639) ↦ InternalError: too much recursion`

Block vs. function scoping

- scoping
 - var: function scoping
 - function f() {
 console.log(x); // for let we get **ReferenceError**
 var x = 10;
 console.log(x);
 return 0;
} // prints undefined, 10 if we use var
 - let: block scoping // new feature

var vs. let in function

var

```
function varTest() {  
  var x = 31;  
  if (true) {  
    var x = 71; // same variable!  
    console.log(x); // 71  
  }  
  console.log(x); // 71  
}
```

let

```
function letTest() {  
  let x = 31;  
  if (true) {  
    let x = 71; // different var.  
    console.log(x); // 71  
  }  
  console.log(x); // 31  
}
```

var vs. let in for cycle

var

```
for (var i = 0; i<10; i++) {  
  console.log(i);  
}  
// 0, 1, 2, 3, 4, ..., 9
```

```
console.log(i);  
// i is 10
```

let

```
for (let i = 0; i<10; i++) {  
  console.log(i);  
}  
// 0, 1, 2, 3, 4, ..., 9
```

```
console.log(i);  
// ReferenceError: i is not  
defined
```

Method chaining (cascading)

- `<div id="d1"></div>`
`<hr></hr>`
- `$("#d1")`
`.html("Hello world!")`
`.css("color", "red")`
`.slideUp(2000).slideDown(2000)`
`.fadeOut().fadeIn();`
- <http://jsfiddle.net/RiOS/9mj4wbeg/>
- design pattern
 - setter methods return the object itself, allowing multiple setters to be invoked in a concise statement
- benefits
 - no need to create temporary variables to save each step of the process
 - “chainable” code can be
 - easily read
 - effortlessly typed

Miscellaneous

- JavaScript is a garbage collected language
- Number is double, no integer data type
- functions arguments are dynamically typed:
 - `function sum(a, b) { return a + b; }`
 - `sum(1, 2); // 3`
 - `sum("a", "b"); // "ab"`
- arguments count does not matter
 - `sum("a"); // "aundefined"`
 - `sum(); // NaN`
 - `function g()`
 - `{ return (Array.apply([], arguments)).join(", "); }`
 - `g("a", 1, 3.14); // "a, 1, 3.14"`

Reserved word usage

- reserved words can not be used in identifiers
 - `var case = 1; // keyword`
 - `SyntaxError: missing variable name`
 - `var class = 1; // future reserved word`
 - `SyntaxError: class is a reserved identifier`
- but can be used in IdentifierName
 - `var o = { case: 1, class: 2 };`
 - `var o = { "case": 1, "class": 2 };`
 - `console.log(o.case, o.class);`
 - `console.log(o["case"], o["class"]);`
- see <http://es5.github.com/#x7.6>

Pattern for default values

- any value can be converted to a boolean
 - false: 0, "", NaN, null, and undefined
 - true: all others values
- `a || "default"`

Immediately-invoked function expression – IIFE

- aka. self executing function: `(function(){...})();`
 - design pattern to give JS missing block scope
 - to avoid cluttering up the current scope, or global scope, with potentially conflicting variables
- Why `function(){}();` doesn't work?
 - `SyntaxError: Unexpected token (`
 - `function(){}` is parsed as `FunctionDeclaration`
 - `(function(){...})` is parsed as `FunctionExpression`
- Why `function(){}(1);` does compile?
 - it is equivalent to: `function(){}; (1);`

Equality operators 1/2

- `===` and `!==` (good ones)
 - the two operands are equal if they are of the **same type** and have the **same value**
- `==` and `!=` (bad ones)
 - the right thing if the operands are of the same type
 - if they are of different types, they attempt to coerce the values $\Rightarrow$   ... (see next slide)
- `Object.is` (new one :- ES6)

Equality operators 2/2

x	y	x == y	x === y	Object.is(x, y)
"	'0'	false	false	false
0	"	true	false	false
0	'0'	true	false	false
false	'false'	false	false	false
false	'0'	true	false	false
false	undefined	false	false	false
false	null	false	false	false
null	undefined	true	false	false
'\t\r\n'	0	true	false	false
+0	-0	true	true	false
NaN	NaN	false	false	true

Overdone implicit typecasting

- `[]+[]`
 - `""`
- `[]+{}`
 - `"[object Object]"`
- `{}+[]`
 - `0`
- `{}+{}`
 - `NaN`
- `"a"+1`
 - `"a1"`
- `"a"-1`
 - `NaN`
- `( (+!![]) + (+!![]) + (+!![]) + (+!![]) + [] )`
 - +
`( (+!![]) + (+!![]) + [] )`
 - `"42"`